

Sara Rinaldo DSC 630 Final Project Code

November 20, 2025

```
[1]: # Load libraries

import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split, \
    StratifiedKFold, GridSearchCV
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (roc_auc_score, average_precision_score, \
    accuracy_score, precision_recall_curve, roc_curve,
    confusion_matrix, classification_report
)
import matplotlib.pyplot as plt
```

```
[11]: # Load data
file_path = "C:\\Users\\czarn\\Documents\\DSC630\\Employee Attrition.csv"
df = pd.read_csv(file_path)
df.head(5)
```

```
[11]:
```

	Age	Attrition	BusinessTravel	Department	DistanceFromHome	\
0	37	Yes	Travel_Rarely	Research & Development	2	
1	21	No	Travel_Rarely	Research & Development	15	
2	45	No	Travel_Rarely	Research & Development	6	
3	23	No	Travel_Rarely	Sales	2	
4	22	No	Travel_Rarely	Research & Development	15	

	Gender	JobInvolvement	JobLevel	JobRole	JobSatisfaction	\
0	Male	2	1	Laboratory Technician	3	
1	Male	3	1	Research Scientist	4	
2	Male	3	3	Research Director	1	
3	Male	3	1	Sales Representative	1	
4	Female	3	1	Laboratory Technician	4	

	Higher_Education	Date_of_Hire	Date_of_termination	
0	Graduation	21-01-2021	NaN	
1	Graduation	13-03-2021	NaN	
2	Post-Graduation	23-01-2021	NaN	
3	PHD	25-04-2021	NaN	
4	PHD	14-06-2021	NaN	

	Status_of_leaving	Mode_of_work	Leaves	Absenteeism	Work_accident	
0	Salary	OFFICE	4	2	No	
1	Work Accident	WFH	5	2	No	
2	Dept.Head	WFH	1	3	No	
3	Work Accident	OFFICE	1	0	Yes	
4	Better Opportunity	WFH	5	2	No	

	Source_of_Hire	Job_mode
0	Job Event	Contract
1	Recruiter	Part Time
2	Job Event	Contract
3	Recruiter	FullTime
4	Job Event	Contract

[5 rows x 32 columns]

```
[3]: # Target variable and data cleaning
df['Attrition'] = (df['Attrition'].str.strip().str.lower() == 'yes').astype(int)
```

```
[4]: # Drop columns that don't have predictive values
for col in ['EmployeeNumber', 'EmployeeCount', 'Over18', 'StandardHours',
            'Date_of_Hire']:
    if col in df.columns:
        df.drop(columns=[col], inplace=True)
```

```
[5]: # Engineered Bands
def make_band(x, bins, labels):
    return pd.cut(x, bins=bins, labels=labels, include_lowest=True)

if 'YearsAtCompany' in df.columns:
    df['TenureBand'] = make_band(
        df['YearsAtCompany'],
        bins=[-np.inf, 2, 5, 10, np.inf],
        labels=['0-2', '2-5', '5-10', '10+']
    )

if 'MonthlyIncome' in df.columns:
    df['IncomeBand'] = pd.qcut(df['MonthlyIncome'], q=4, labels=['Q1', 'Q2',
        'Q3', 'Q4'])
```

```
y = df['Attrition'].values
X = df.drop(columns=['Attrition'])
```

```
[8]: # Identify types
numeric_cols = X.select_dtypes(include=[np.number]).columns.tolist()
categorical_cols = X.select_dtypes(exclude=[np.number]).columns.tolist()
```

```
[9]: # Preprocessor to remove missing values, handle numeric and categorical columns
numeric_pipe = Pipeline([
    ('impute', SimpleImputer(strategy='median')),
    ('scale', StandardScaler())
])

categorical_pipe = Pipeline([
    ('impute', SimpleImputer(strategy='most_frequent')),
    ('one', OneHotEncoder(handle_unknown='ignore'))
])

preprocess = ColumnTransformer([
    ('num', numeric_pipe, numeric_cols),
    ('cat', categorical_pipe, categorical_cols)
])
```

```
[11]: # Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=42
)
```

```
[19]: # Baseline Logistic Regression
logit = Pipeline([
    ('pre', preprocess),
    ('clf', LogisticRegression(max_iter=5000, class_weight='balanced',
    ↪ solver='saga'))
])

logit_grid = {
    'clf__C': [0.1, 0.5, 1.0, 2.0, 5.0],
    'clf__penalty': ['l2']
}

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
logit_gs = GridSearchCV(logit, logit_grid, cv=cv, scoring='roc_auc', n_jobs=-1)
logit_gs.fit(X_train, y_train)
```

```
[19]: GridSearchCV(cv=StratifiedKFold(n_splits=5, random_state=42, shuffle=True),
    estimator=Pipeline(steps=[('pre',
```

```

ColumnTransformer(transformers=[('num',
Pipeline(steps=[('impute',
                  SimpleImputer(strategy='median')),
                  ('scale',
                   StandardScaler())])),
                                ('cat',
Pipeline(steps=[('impute',
                  SimpleImputer(strategy='most_frequent')),
                  ('one_hot',
                   OneHotEncoder(handle_unknown='ignore'))])),
                                ('Age',
Pipeline(steps=[('impute',
                  SimpleImputer(strategy='most_frequent'))])),
                                ('DistanceFromHome',
Pipeline(steps=[('impute',
                  SimpleImputer(strategy='most_frequent'))])),
                                ('JobInvolvement',
Pipeline(steps=[('impute',
                  SimpleImputer(strategy='most_frequent'))])),
                                ('JobLevel',
Pipeline(steps=[('impute',
                  SimpleImputer(strategy='most_frequent'))])),
                                ('JobSatisfaction',
Pipeline(steps=[('impute',
                  SimpleImputer(strategy='most_frequent'))])),
                                ('MonthlyIncome',
Pipeline(steps=[('impute',
                  SimpleImputer(strategy='most_frequent'))])),
                                ('NumCompaniesWorked',
Pipeline(steps=[('impute',
                  SimpleImputer(strategy='most_frequent'))])),
                                ('Percen...',
Pipeline(steps=[('impute',
                  SimpleImputer(strategy='most_frequent'))])),
                                ('Gender',
Pipeline(steps=[('impute',
                  SimpleImputer(strategy='most_frequent'))])),
                                ('JobRole',
Pipeline(steps=[('impute',
                  SimpleImputer(strategy='most_frequent'))])),
                                ('MaritalStatus',
Pipeline(steps=[('impute',
                  SimpleImputer(strategy='most_frequent'))])),
                                ('OverTime',
Pipeline(steps=[('impute',
                  SimpleImputer(strategy='most_frequent'))])),
                                ('Higher_Education',
Pipeline(steps=[('impute',
                  SimpleImputer(strategy='most_frequent'))])),
                                ('Status_of_leaving',
Pipeline(steps=[('impute',
                  SimpleImputer(strategy='most_frequent'))])),
                                ('Mode_of_work',
Pipeline(steps=[('impute',
                  SimpleImputer(strategy='most_frequent'))])),
                                ('Work_accident',
Pipeline(steps=[('impute',
                  SimpleImputer(strategy='most_frequent'))])),
                                ('Source_of_Hire',
Pipeline(steps=[('impute',
                  SimpleImputer(strategy='most_frequent'))])),
                                ('Job_mode',
Pipeline(steps=[('impute',
                  SimpleImputer(strategy='most_frequent'))])),
                                ('TenureBand',
Pipeline(steps=[('impute',
                  SimpleImputer(strategy='most_frequent'))])),
                                ('IncomeBand']]))),
                                ('clf',
LogisticRegression(class_weight='balanced',
                    max_iter=5000,
                    solver='saga'))),
n_jobs=-1,
param_grid={'clf__C': [0.1, 0.5, 1.0, 2.0, 5.0],
            'clf__penalty': ['l2']},
scoring='roc_auc')

```

```

[20]: rf = Pipeline([
        ('pre', preprocess),
        ('clf', RandomForestClassifier(class_weight='balanced', random_state=42))
    ])

rf_grid = {
    'clf__n_estimators': [300, 600],
    'clf__max_depth': [None, 6, 10],
    'clf__min_samples_split': [2,5],
    'clf__min_samples_leaf': [1,2]
}

rf_gs = GridSearchCV(rf, rf_grid, cv=cv, scoring='roc_auc', n_jobs=-1)

```

```
rf_gs.fit(X_train, y_train)
```

```
[20]: GridSearchCV(cv=StratifiedKFold(n_splits=5, random_state=42, shuffle=True),
                  estimator=Pipeline(steps=[('pre',
                                             ColumnTransformer(transformers=[('num',
Pipeline(steps=[('impute',
                  SimpleImputer(strategy='median')),
                  ('scale',
                    StandardScaler()))]),
                                             ['Age',

'DistanceFromHome',
'JobInvolvement',
'JobLevel',
'JobSatisfaction',
'MonthlyIncome',
'NumCompaniesWorked',
'Percen...
'Higher_Education',
'Status_of_leaving',
'Mode_of_work',
'Work_accident',
'Source_of_Hire',
'Job_mode',
'TenureBand',
'IncomeBand'])])),
                                             ('clf',
RandomForestClassifier(class_weight='balanced',
random_state=42))]),
                  n_jobs=-1,
                  param_grid={'clf__max_depth': [None, 6, 10],
                              'clf__min_samples_leaf': [1, 2],
                              'clf__min_samples_split': [2, 5],
                              'clf__n_estimators': [300, 600]},
                  scoring='roc_auc')
```

```
[21]: # Evaluate on test set
def eval_and_plot(best_pipeline, name):
    proba = best_pipeline.predict_proba(X_test)[: , 1]
    preds = (proba >= 0.5).astype(int)

    roc_auc = roc_auc_score(y_test, proba)
    pr_auc = average_precision_score(y_test, proba)
    acc = accuracy_score(y_test, preds)
    cm = confusion_matrix(y_test, preds)

    print(f'\n=== {name} ===')
```

```

print(f'ROC-AUC: {roc_auc:.3f} | PR-AUC: {pr_auc:.3f} | Accuracy: {acc:.3f}')
print('Confusion matrix:\n', cm)
print('\nClassification report:\n', classification_report(y_test, preds, digits=3))

# ROC Curve

fpr, tpr, _ = roc_curve(y_test, proba)
plt.figure()
plt.plot(fpr, tpr, label=f'{name} (AUC={roc_auc:.3f})')
plt.plot([0,1],[0,1], linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title(f'ROC Curve - {name}')
plt.legend(loc='lower right')
plt.show()

# Precision Recall Curve

prec, rec, _ = precision_recall_curve(y_test, proba)
plt.figure()
plt.plot(rec, prec, label=f'{name} (PR-AUC={pr_auc:.3f})')
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title(f'Precision-Recall Curve - {name}')
plt.legend(loc='lower left')
plt.show()

eval_and_plot(logit_gs.best_estimator_, 'Logistic Regression')
eval_and_plot(rf_gs.best_estimator_, 'Random Forest')

```

=== Logistic Regression ===

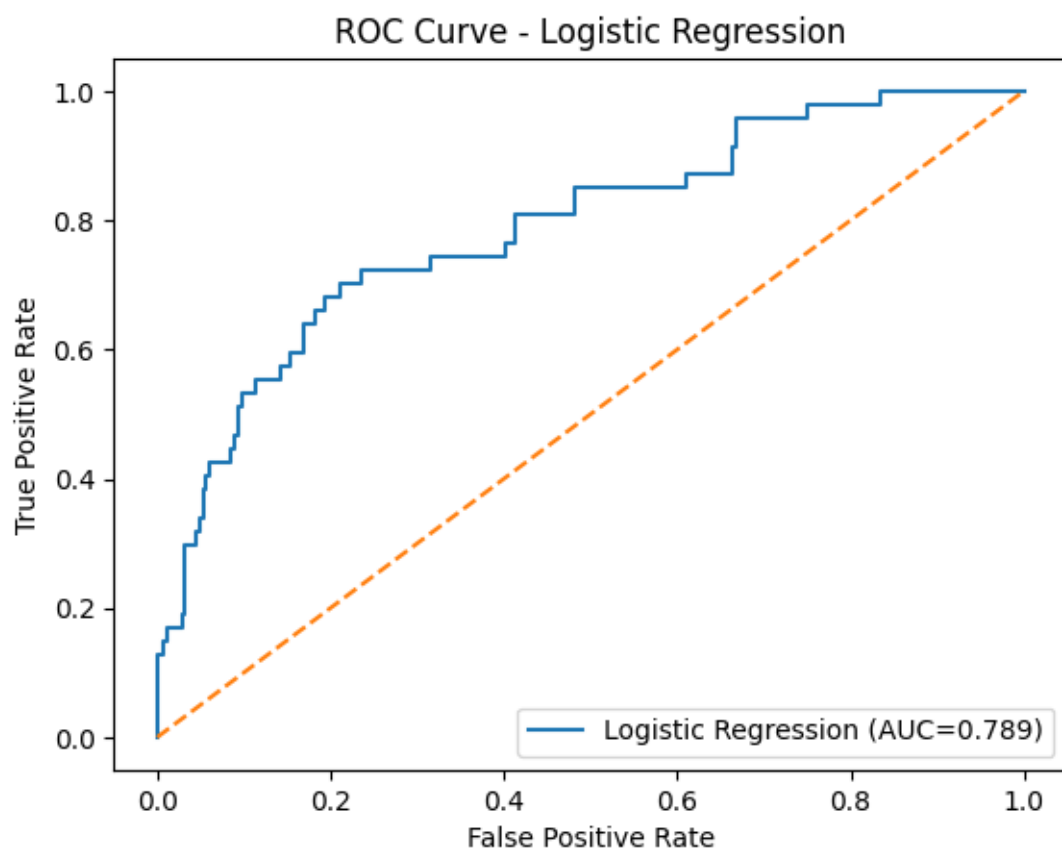
ROC-AUC: 0.789 | PR-AUC: 0.506 | Accuracy: 0.793

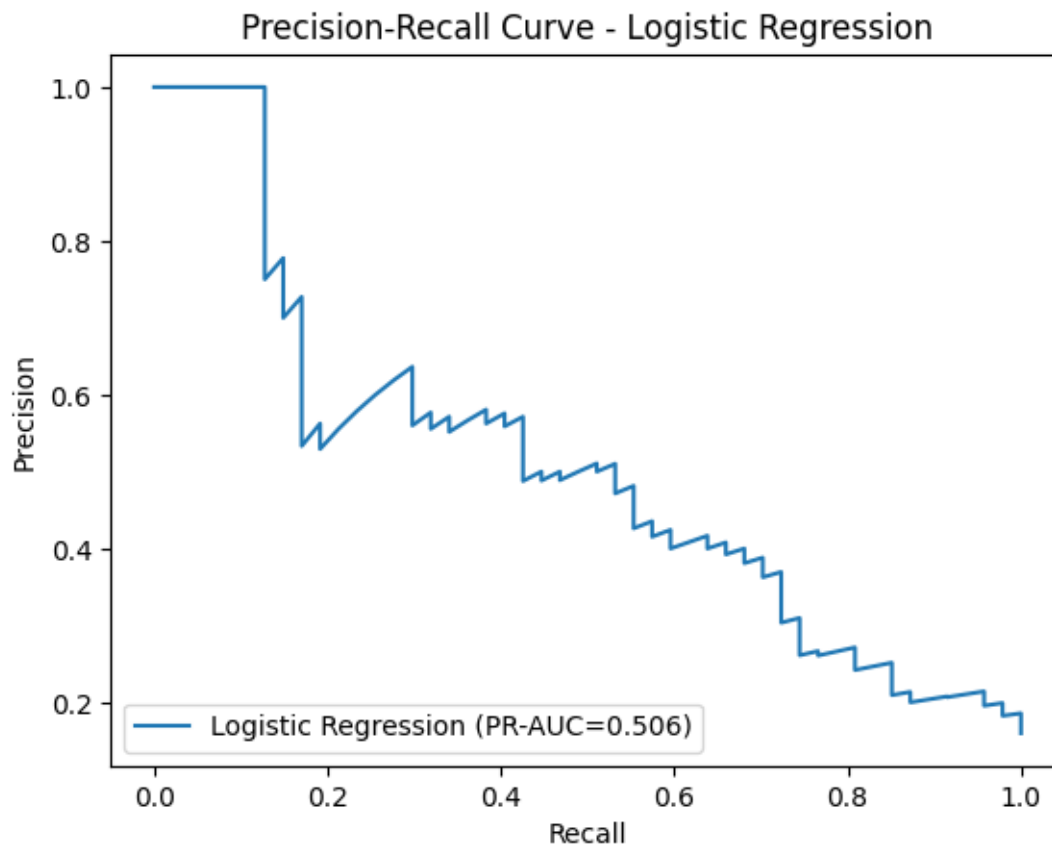
Confusion matrix:

```
[[203  44]
 [ 17  30]]
```

Classification report:

	precision	recall	f1-score	support
0	0.923	0.822	0.869	247
1	0.405	0.638	0.496	47
accuracy			0.793	294
macro avg	0.664	0.730	0.683	294
weighted avg	0.840	0.793	0.810	294





=== Random_Forest ===

ROC-AUC: 0.758 | PR-AUC: 0.507 | Accuracy: 0.864

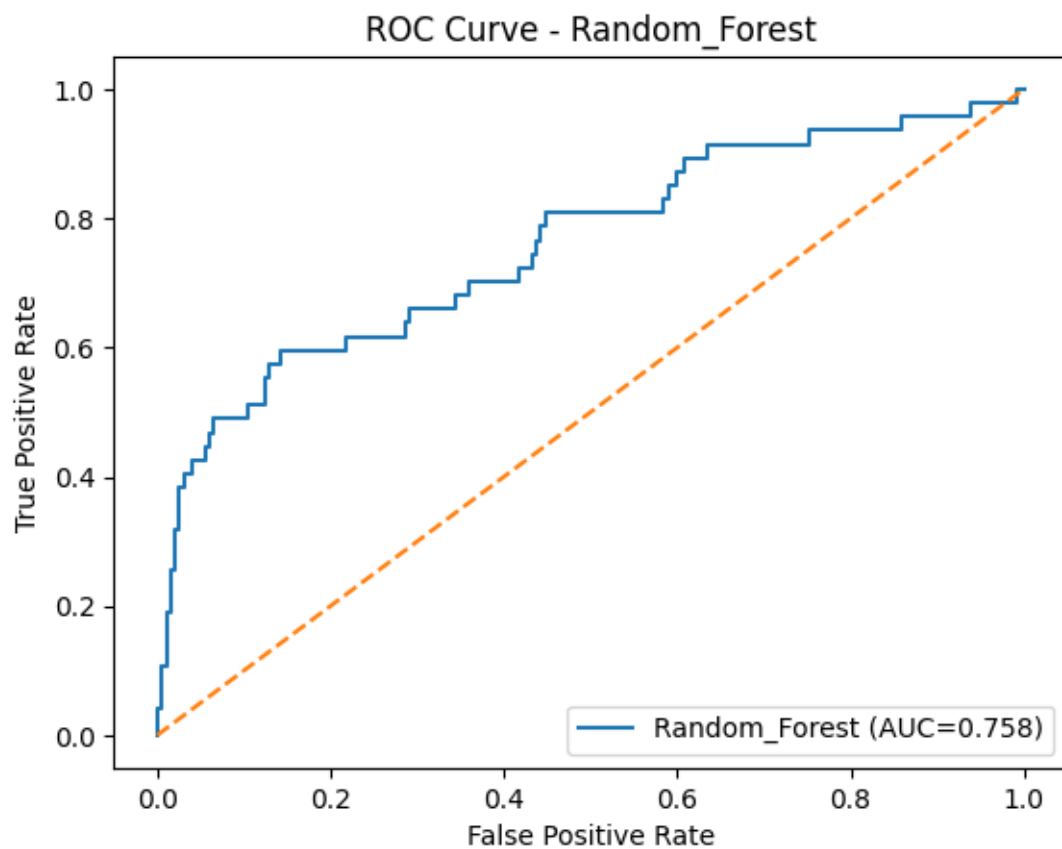
Confusion matrix:

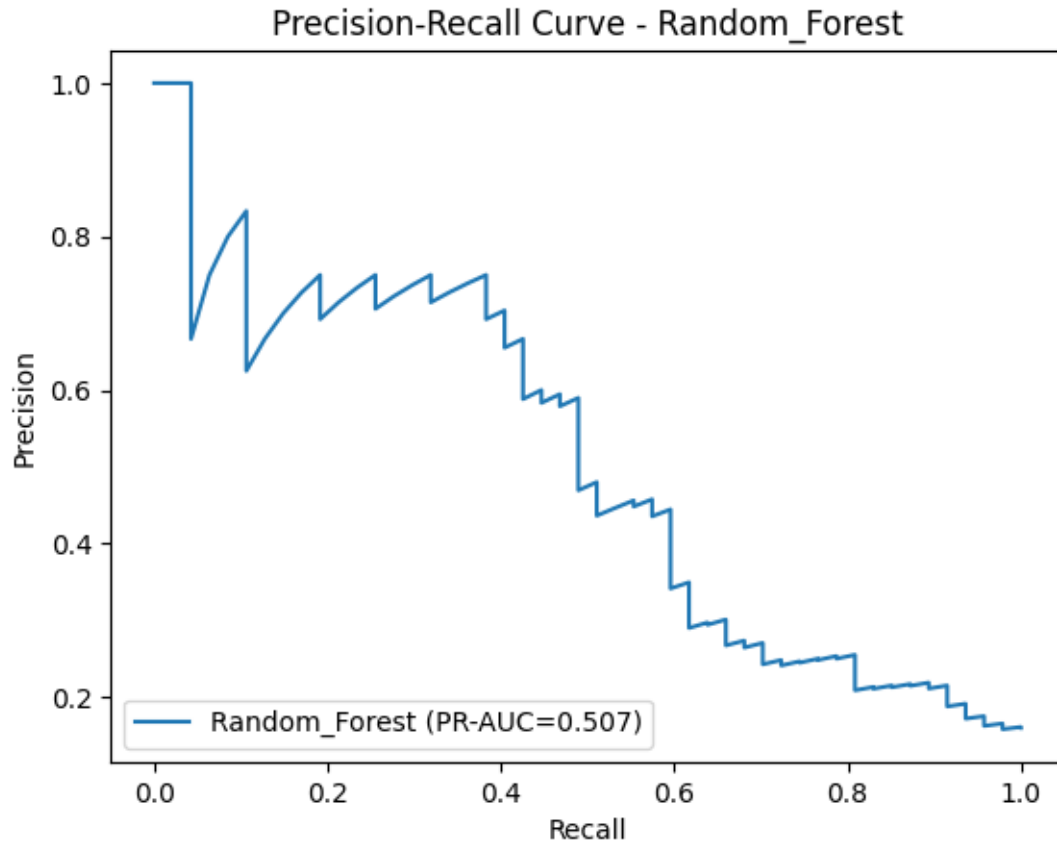
```
[[242  5]
```

```
[ 35 12]]
```

Classification report:

	precision	recall	f1-score	support
0	0.874	0.980	0.924	247
1	0.706	0.255	0.375	47
accuracy			0.864	294
macro avg	0.790	0.618	0.649	294
weighted avg	0.847	0.864	0.836	294





```
[30]: # Feature importance/ coefficients
logit_best = logit_gs.best_estimator_
feature_names = (
    logit_best.named_steps['pre']
    .get_feature_names_out()
)

if hasattr(logit_best.named_steps['clf'], 'coef_'):
    coefs = logit_best.named_steps['clf'].coef_[0]
    top_idx = np.argsort(coefs)
    print('\nTop negative LR coefficients:')
    for i in top_idx[:10]:
        print(f'{feature_names[i]}: {coefs[i]:.3f}')
    print('\nTop positive LR coefficients:')
    for i in top_idx[-10:][::-1]:
        print(f'{feature_names[i]}: {coefs[i]:.3f}')
```

Top negative LR coefficients:
cat__OverTime_No: -0.730

```
cat__BusinessTravel_Non-Travel: -0.721
num__YearsWithCurrManager: -0.425
cat__Department_Research & Development: -0.384
num__JobInvolvement: -0.349
num__JobSatisfaction: -0.347
cat__Job_mode_Part Time: -0.345
cat__MaritalStatus_Divorced: -0.336
cat__JobRole_Manager: -0.315
cat__TenureBand_2-5: -0.314
```

Top positive LR coefficients:

```
cat__OverTime_Yes: 0.724
cat__BusinessTravel_Travel_Frequently: 0.633
cat__JobRole_Laboratory Technician: 0.551
num__YearsAtCompany: 0.527
cat__TenureBand_0-2: 0.483
cat__MaritalStatus_Single: 0.371
cat__Department_Sales: 0.349
num__DistanceFromHome: 0.337
cat__IncomeBand_Q1: 0.336
cat__Job_mode_Contract: 0.326
```

```
[23]: # Random Forest importances
rf_best = rf_gs.best_estimator_
if hasattr(rf_best.named_steps['clf'], 'feature_importances_'):
    importances = rf_best.named_steps['clf'].feature_importances_
    idx = np.argsort(importances)[-15:]
    plt.figure()
    plt.barh(range(len(idx)), importances[idx])
    plt.yticks(range(len(idx)), [feature_names[i] for i in idx])
    plt.xlabel('Importance')
    plt.title('Top Feature Importances - Random Forest')
    plt.tight_layout()
    plt.show()
```

